List of Experiments

Total No of Labs: 14

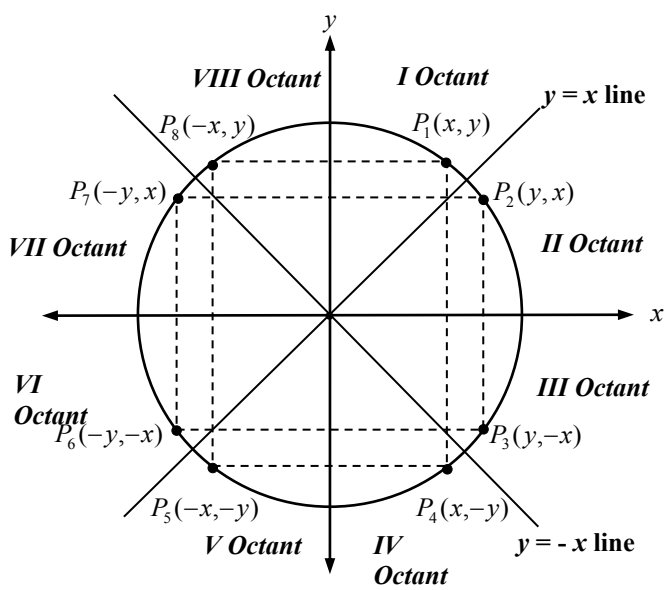
S. No.	Programs List	No. of Labs
CAD Experiments		
1	Study of basic graphics functions defined in “graphics.h”.	1
2	To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).	1
3	To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).	1
4	To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.	1
5	To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.	2
6	To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.	2
7	To generate a smooth curve by using Bezier curve technique for a given set of control points.	1
8	To implement Cohen–Sutherland 2D clipping and window–viewport mapping	1
Product Design		
9	Introduction to CAD software’s.	1
10	Generation of any one simple solid model using any CAD software.	1
11	Generation of any one assembly of mechanical part using any CAD software.	1
12	Design and develop the complete lifecycle for any product.	1

Experiment - 4

Aim: To implement Bresenham's Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.

Description:

Bresenham's algorithm extensively uses symmetry of the circle. This is fact that if $\frac{1}{8}$ part of the circle is generated then rest $\frac{7}{8}$ part of the circle can be generated using the symmetry, as shown in Fig. 3.16. If (x, y) coordinates of a point P_1 in the first octant is calculated, it is easy to find out the remaining seven points in the other seven octants such as $P_2(y,x)$, $P_3(y,-x)$, $P_4(x,-y)$, $P_5(-x,-y)$, $P_6(-y,-x)$, $P_7(-y,x)$ and $P_8(-x,y)$. Therefore, in Bresenham's midpoint circle generating algorithm, it is required to find out pixel positions only in the region of $\frac{1}{8}$ part of the circle (i.e., one octant), and hence, development of software is very easy. Moreover, the circle draws at a faster rate.



In this method, at first, the topmost point, i.e., $(0,r)$ is plotted if equation of circle is $x^2 + y^2 = r^2$. However, for circle with centre position (x_c, y_c) and radius r, the algorithm can be developed for the calculation of pixel positions around the circle path, centered at origin $(0,0)$. Thereafter, each evaluated pixel position (x,y) is shifted to its proper screen position by adding x_c

to x and y_c to y along circle section in the first octant (from $x = 0$ line to $x = y$ line; alternatively, when slope varies from $m = 0$ to $m = -1$). Therefore, we can go in the positive x-direction by unit steps over the first octant. When x coordinate of a point increases, corresponding y coordinate decreases.

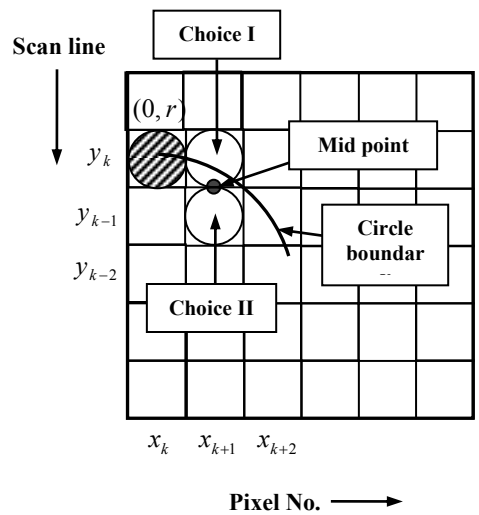
Similar to other Bresenham's algorithms, a decision parameter P_k determines the two possible pixel positions, which is closer to the actual circle boundary at each step in the first octant. Pixel positions in the other seven octants are determined using the symmetry of circle. Let circle function itself is the decision parameter in the Bresenham's algorithm. Thus, the equation of circle in implicit form is

$$P_k = f_{circle}(x, y) = x^2 + y^2 - r^2 \begin{cases} < 0, \text{ if point } (x, y) \text{ lies inside the circle boundary} \\ = 0, \text{ if point } (x, y) \text{ lies on the circle boundary} \\ > 0, \text{ if point } (x, y) \text{ lies outside the circle boundary} \end{cases}$$

Suppose, Fig. 3.17 shows the plot for the current pixel coordinates (x_k, y_k). Now, it is to decide that at the sampling position x_{k+1}, whether the pixel y_k or one at y_{k-1} is closer to the circle boundary. Alternatively, at position x_{k+1}, we have to select either pixel coordinate position (x_{k+1}, y_k) as choice I or pixel position (x_{k+1}, y_{k-1}) as choice II, depending upon position of the midpoint between two adjacent pixels y_k and y_{k-1} from the circle boundary. The decision parameter P_k at the midpoint between these two pixels will be

$$P_k = f_{circle}(x_{k+1}, \frac{y_k + y_{k-1}}{2}) = f_{circle}(x_k + 1, y_k - \tfrac{1}{2}) \qquad (1)$$

$$= (x_k + 1)^2 + (y_k - \tfrac{1}{2})^2 - r^2$$



Now, there are two possibilities for the decision parameter P_k :

(i). If $P_k < 0$, the midpoint lies inside the circle; alternatively, pixel on the scan line y_k is closer to the circle boundary and pixel choice will be (x_{k+1}, y_k), i.e., choice I.

(ii). If $P_k \geq 0$, the midpoint lies outside or on the circle boundary; alternatively, pixel on the scan line y_{k-1} becomes closer to the circle boundary and pixel choice will be (x_{k+1}, y_{k-1}), i.e., choice II.

The recursive formula for successive values of the decision parameter P_k may be developed by using the incremental calculations for the next pixel position at $x_{k+1} + 1 = x_k + 2$. Thus, recursive expression for P_{k+1} may be written as

$$\begin{aligned} P_{k+1} &= f_{circle}(x_{k+1}+1, y_{k+1} - \tfrac{1}{2}) \\ &= \{(x_k+1)^2 + 1\}^2 + \{y_{k+1} - \tfrac{1}{2}\}^2 - r^2 \\ &= (x_k+1)^2 + 1 + 2.(x_k+1) + y_{k+1}^2 + \tfrac{1}{4} - y_{k+1} - r^2 \\ &= \{(x_k+1)^2 + (y_k^2 + \tfrac{1}{4} - y_k) - r^2\} + 1 + 2.(x_k+1) + y_{k+1}^2 - y_{k+1} - y_k^2 + y_k \\ &= \{(x_k+1)^2 + (y_k^2 + \tfrac{1}{4} - y_k) - r^2\} + 1 + 2.(x_k+1) + y_{k+1}^2 - y_{k+1} - y_k^2 + y_k \end{aligned}$$

or $$P_{k+1} = P_k + 2.(x_k+1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1 \qquad (2)$$

where $y_{k+1} = y_k$ (if $P_k < 0$, i.e., negative) when pixel (x_{k+1}, y_k) lies inside the circle boundary; therefore, $P_{k+1} = P_k + 2.x_{k+1} + 1$ (3)

and $y_{k+1} = y_{k-1}$ (if $P_k \geq 0$, i.e., positive) when pixel (x_{k+1}, y_{k-1}) lies outside the circle boundary; therefore, $y_{k+1} = y_{k-1} = y_k - 1$ gives $y_{k+1} - y_k = -1$ and $y_{k+1}^2 - y_k^2 = 1 - 2y_k$

moreover, the decision parameter may be written as

$$P_{k+1} = P_k + 2.x_{k+1} + 1 - 2.(y_k - 1)$$

$$= P_k + 2.x_{k+1} + 1 - 2.y_{k+1} \quad \text{(because } y_{k+1} = y_k - 1\text{)} \quad (4)$$

From eqn. (3.13), the initial value of decision parameter at topmost point $(0, r)$, calculated as

$$P_0 = f_{circle}(1, r - \tfrac{1}{2}) = 1 + (r - \tfrac{1}{2})^2 - r^2 = \tfrac{5}{4} - r \quad (5)$$

When radius of the circle is specified as integer value, P_0 may be modified as

$$P_0 \cong 1 - r$$

Algorithm

1. Start.
2. Declare variables x,y,p and also declare gdriver=DETECT,gmode.
3. Initialise the graphic mode with the path location in TC folder.
4. Input the radius of the circle r.
5. Load x-0,y=r,initial decision parameter p=1-r.so the first point is (0,r).
6. Repeat Step 7 while (x<y) and increment x-value simultaneously.
7. If (p>0),do p=p+2*(x-y)+1.
    a. Otherwise p=p+2*x+1 and y is decremented simultaneously.
8. Then calculate the value of the function circlepoints() with parameters (x,y).
9. Place pixels using putpixel at points (x+300,y+300) in specified colour in circlepoints() function shifting the origin to 300 on both x-axis and y-axis.
10. Close Graph.
11. Stop.

<u>Programe</u>

```
# include<stdio.h>
# include<graphics.h>
```

```
# include<conio.h>
# include<math.h>

void bresenham_circle(const int h,const int k,const int r)
{
//    cleardevice();
    int x=0,y=r,p=(3-(2*r));
    cleardevice();
    line(320,1,320,480);
    line(1,240,640,240);
    do
          {
             putpixel((h+x),(k+y),15);
             putpixel((h+y),(k+x),15);
             putpixel((h+y),(k-x),15);
             putpixel((h+x),(k-y),15);
             putpixel((h-x),(k-y),15);
             putpixel((h-y),(k-x),15);
             putpixel((h-y),(k+x),15);
             putpixel((h-x),(k+y),15);
       x++;
       if(p<0)
                    p+=((4*x)+6);
       else
                    {
         y--;
                      p+=((4*(x-y))+10);
                    }
          }
    while(x<=y);
}

void main(void)
{
    int driver=VGA,mode=VGAHI,h,k,r;
    initgraph(&driver,&mode,"..\\Bgi");
    printf("\n ENTER THE VALUE OF [X- COORDINATE]      :");
    scanf("%d",&h);
    printf("\n ENTER THE VALUE OF [Y- COORDINATE]      :");
    scanf("%d",&k);
    printf("\n ENTER THE VALUE OF RADIUS               :");
    scanf("%d",&r);
```

```
    bresenham_circle(320+h,240-k,r);
getch();
}
```